

# Natural Language Processing (NLP)

AI Engineer Placement Preparation Series

Generated: February 2026 | Transformer Era Edition

# Table of Contents

1. Why NLP Matters (2025 Reality)
2. Text Preprocessing Fundamentals
3. Classic Text Representations
4. Word Embeddings (Word2Vec, GloVe, FastText)
5. Classical NLP Tasks
6. The Attention Mechanism
7. Transformers Architecture
8. BERT & Encoder Models
9. GPT & Decoder Models
10. Hugging Face Ecosystem
11. Fine-Tuning & Practical Workflows
12. Common Mistakes
13. Quick-Fire Q&A;
14. One-Line Cheat Sheet

## 1. Why NLP Matters (2025 Reality)

NLP has undergone a seismic shift. Five years ago, mastering text preprocessing, TF-IDF, and classical models like SVM was sufficient. Today? ChatGPT, Claude, Gemini, Llama—all are transformer-based language models. Every major tech company is hiring AI Engineers specifically for LLM work. The 2025 expectation is clear:

You must understand transformers deeply. You must know how to fine-tune BERT or use GPT APIs. You must understand prompt engineering, attention mechanisms, and why BERT is bidirectional while GPT is autoregressive. Classic NLP is still relevant (preprocessing, classical tasks) but transformers are the main event.

**This module structure:** We'll build from fundamentals (text preprocessing) to classical approaches (embeddings, basic tasks) to cutting-edge (transformers, BERT, fine-tuning). You'll understand the full evolution of NLP and why we got here.

## 2. Text Preprocessing Fundamentals

### 2.1 Why Preprocessing Matters

Raw text is messy: inconsistent casing, punctuation, URLs, HTML tags, special characters. Models learn better from clean, consistent input. Preprocessing reduces noise and shrinks vocabulary size (fewer unique tokens to learn).

### 2.2 Core Preprocessing Steps

```
# Example text
text = "Hello, this is SAMPLE text! Visit http://example.com for more."

# 1. Lowercasing
text = text.lower()
# "hello, this is sample text! visit http://example.com for more."

# 2. Remove URLs
import re
text = re.sub(r'http\S+|www\S+', '', text)

# 3. Remove special characters & punctuation
text = re.sub(r'^a-zA-Z0-9\s', '', text)
# "hello this is sample text visit examplecom for more"

# 4. Tokenization (split into words)
tokens = text.split()
# ['hello', 'this', 'is', 'sample', 'text', 'visit', ...]

# 5. Remove stopwords (common words with little meaning)
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
tokens = [w for w in tokens if w not in stop_words]
# ['hello', 'sample', 'text', 'visit', ...]
```

### 2.3 Tokenization Methods

**Word Tokenization:** Split by whitespace/punctuation. Simple but ignores subwords. 'unbelievable' is 1 token.

**Character Tokenization:** Split into characters. Handles OOV (out-of-vocabulary) but creates long sequences.

**Subword Tokenization:** BPE (Byte Pair Encoding), WordPiece, SentencePiece. Balance between word and character. 'unbelievable' → ['unbelieve', 'able']. Used in transformers (BERT uses WordPiece, GPT uses BPE). Handles OOV + keeps sequences manageable.

### 2.4 Stemming vs Lemmatization

| Method            | Example              | How It Works                | Pros          | Cons                     |
|-------------------|----------------------|-----------------------------|---------------|--------------------------|
| Stemming (Porter) | running, runs → runn | Remove suffixes (heuristic) | Fast & simple | Doesn't handle all cases |

|               |                     |                           |                         |                             |
|---------------|---------------------|---------------------------|-------------------------|-----------------------------|
| Lemmatization | running, runs → run | Uses dictionary + grammar | <del>and (slower)</del> | Slower, requires NLTK/spaCy |
|---------------|---------------------|---------------------------|-------------------------|-----------------------------|

**Note:** Modern transformers (BERT, GPT) use subword tokenization + learned embeddings. You rarely need stemming/lemmatization. But for classical ML (TF-IDF + SVM), they help.

### 3. Classic Text Representations

#### 3.1 Bag of Words (BoW)

Represent text as a vector of word frequencies. Order doesn't matter (hence "bag"). Each unique word is a dimension.

```
# Two sentences:
s1 = "cat sat on mat"
s2 = "dog sat on log"

# Vocabulary: {cat, sat, on, mat, dog, log}
# s1 vector: [1, 1, 1, 1, 0, 0] (cat, sat, on, mat, dog, log)
# s2 vector: [0, 1, 1, 0, 1, 1]

# Cosine similarity = dot product / (||v1|| * ||v2||)
# Score = 2 / (2 * 1.73) ≈ 0.58 (moderately similar)
```

#### 3.2 TF-IDF (Term Frequency - Inverse Document Frequency)

Weights words by how important they are to a document. Common words (the, is) get low weight. Rare words get high weight. Better than BoW because it captures importance.

```
TF-IDF(w, d) = TF(w, d) * IDF(w)

TF(w, d) = (count of w in d) / (total words in d)
           → How often w appears in d

IDF(w) = log(total docs / docs with w)
         → Rarity of w across corpus

Example: "the" appears in 90% of docs → IDF(the) is low
         "transformer" appears in 2% of docs → IDF(transformer) is high
```

#### 3.3 Python Implementation

```
from sklearn.feature_extraction.text import TfidfVectorizer

texts = [
    "I love deep learning",
    "Deep learning is amazing",
    "I hate shallow learning"
]

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(texts)
# shape: (3, 6) – 3 docs, 6 unique words
# sparse matrix (mostly zeros)

# Get feature names
vectorizer.get_feature_names_out()
# array(['amazing', 'deep', 'hate', 'i', 'learning', 'love', 'shallow'], ...)
```

**Limitation of BoW & TF-IDF:** No semantic understanding. "cat" and "dog" are equally distant. No notion of "dog is like cat". This is where word embeddings come in.

## 4. Word Embeddings (Word2Vec, GloVe, FastText)

### 4.1 The Key Insight

Instead of sparse one-hot vectors, learn dense vectors (embeddings) where similar words have similar vectors. "king" and "queen" should be close. "king - man + woman  $\approx$  queen" (semantic analogy). Embeddings capture semantic meaning, not just frequency.

### 4.2 Word2Vec

Two training approaches. Both use a neural network to learn embeddings by predicting words from context (or vice versa).

| Method                            | Task                             | Explanation                          | When to Use                      |
|-----------------------------------|----------------------------------|--------------------------------------|----------------------------------|
| CBOW<br>(Continuous Bag of Words) | Predict center word from context | Given [cat, on, mat], predict "sat". | Fast training. Small vocab.      |
| Skip-gram                         | Predict context from center word | Given "sat", predict [cat, on, mat]. | Better quality. Larger datasets. |

### 4.3 GloVe (Global Vectors)

Matrix factorization approach. Uses global word co-occurrence statistics. Combines benefits of count-based (LSA) and prediction-based (Word2Vec) methods. Often performs better than Word2Vec.

### 4.4 FastText

Extension of Word2Vec. Learns embeddings for subwords (character n-grams), not just full words. Handles OOV words and morphology better. Example: "running" = emb(run) + emb(ing). Great for morphologically rich languages.

### 4.5 Using Pre-trained Embeddings

```
import gensim.downloader as api

# Download pre-trained Word2Vec (trained on Google News)
w2v = api.load('word2vec-google-news-300')

# Access embeddings
embedding = w2v['king'] # shape: (300,)

# Semantic operations
similar = w2v.most_similar('king', topn=5)
# [('queen', 0.645), ('monarch', 0.634), ...]

# Analogy: king - man + woman = ?
w2v.most_similar(positive=['king', 'woman'], negative=['man'])
# [('queen', 0.715), ...]

# Similarity between two words
similarity = w2v.similarity('cat', 'dog') # ~0.80
```

## 5. Classical NLP Tasks

### 5.1 Text Classification

Assign text to predefined categories. Example: sentiment analysis (positive/negative/neutral), spam detection (spam/not spam), topic classification (news categories).

```
# Approach 1: TF-IDF + SVM (classical)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import SVC

texts = ["I love this movie", "This is terrible", ...]
labels = [1, 0, ...] # 1=positive, 0=negative

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)
model = SVC()
model.fit(X, labels)

# Approach 2: Use pre-trained transformer
from transformers import pipeline
classifier = pipeline('sentiment-analysis')
result = classifier('I love deep learning')[0]
# {'label': 'POSITIVE', 'score': 0.999}
```

### 5.2 Named Entity Recognition (NER)

Extract entities (names, locations, organizations) from text. Example: "Apple CEO Tim Cook announced..." → {Apple: ORG, Tim Cook: PERSON}. Classical approach: sequence labeling (each token gets a label: PERSON, ORG, LOC, O).

```
# Using spaCy (trained pipeline)
import spacy

nlp = spacy.load('en_core_web_sm')
text = "Apple CEO Tim Cook announced new iPhone"
doc = nlp(text)

for ent in doc.ents:
    print(f"{ent.text} → {ent.label_}")
# Apple → ORG
# Tim Cook → PERSON
# iPhone → PRODUCT
```

### 5.3 Part-of-Speech (POS) Tagging

Label each word with its grammatical role: NOUN, VERB, ADJ, ADV, etc. Example: "The cat sat" → DET NOUN VERB. Useful for parsing, information extraction.

### 5.4 Topic Modeling (LDA)

Discover hidden topics in a document collection. Latent Dirichlet Allocation (LDA) assumes each document is a mixture of topics, and each topic is a mixture of words. Less used now (transformers are better), but still important to know.

```
from gensim import corpora, models

# Prepare documents (tokenized lists)
documents = [["deep", "learning"], ["machine", "learning"], ...]

# Create dictionary and corpus
dictionary = corpora.Dictionary(documents)
corpus = [dictionary.doc2bow(doc) for doc in documents]

# Train LDA (k=3 topics)
lda = models.LdaModel(corpus=corpus, id2word=dictionary, num_topics=3)

# Print topics
for idx, topic in lda.print_topics(num_words=3):
    print(f"Topic {idx}: {topic}")
# Topic 0: 0.05*deep + 0.04*learning + 0.03*network
# Topic 1: 0.06*data + 0.05*analysis + 0.04*statistics
# ...
```

## 6. The Attention Mechanism (The Game Changer)

### 6.1 The Problem It Solves

Sequence-to-sequence models (encoder-decoder RNNs) struggle with long sequences. The encoder compresses the entire input into a single vector (bottleneck). The decoder then generates output from this vector. For long sequences, information is lost.

Attention allows the decoder to "look back" at all encoder hidden states and decide which parts of the input are relevant for each output token. Solves bottleneck and long-dependency problem.

### 6.2 Self-Attention (Scaled Dot-Product Attention)

Each token attends to all other tokens in the same sequence. Compute relevance scores ("which words matter?") for each position.

```
# Scaled dot-product attention
# Input: Query (Q), Key (K), Value (V) matrices [seq_len, d_model]

Attention(Q, K, V) = softmax(Q·KT / √dk) · V

# Explanation:
# Q·KT: [seq_len, seq_len] matrix of relevance scores
# √dk: scaling factor (prevents softmax from saturating)
# softmax(...): normalize to [0, 1] (attention weights)
# (...) · V: weighted combination of values

# Example: "The cat sat on the mat"
# For token "sat", query attends to all words
# Attention weights: [0.05, 0.15, 0.5, 0.15, 0.05, 0.1]
#           the   cat   sat   on   the   mat
# Output: 0.5*sat_embedding + 0.15*cat_embedding + ... (weighted sum)
```

### 6.3 Multi-Head Attention

Not one attention mechanism, but multiple (8, 12, 16 heads). Each head learns different aspects (some focus on syntax, some on semantics). Concatenate outputs. More powerful, captures diverse patterns.

```
# Multi-head attention (8 heads)
# For each head:
#   - Project Q, K, V to smaller dimensions (d_model / 8)
#   - Compute attention
#   - Back-project

# Concatenate all 8 outputs
# Final linear transformation

# Result: Attention(Q, K, V) ← 8 different perspectives simultaneously
```

# 7. Transformers Architecture

## 7.1 Overview: "Attention Is All You Need"

The 2017 paper that started the revolution. Key idea: No RNNs or CNNs. Just attention + feedforward networks. Parallelizable (unlike RNNs which process sequentially). Scales to large datasets. This is the foundation of GPT, BERT, all modern LLMs.

## 7.2 Components

**Positional Encoding:** Attention has no notion of position (no recurrence). Add position information via sinusoidal functions to embeddings. Token at position 0 gets one encoding, position 1 gets another. Model learns relative positions.

**Encoder Stack:** Multiple (6 in original) identical blocks. Each block: Multi-head self-attention + Feed-forward network + Layer normalization + Residual connections. Encoder output: contextualized representation of input.

**Decoder Stack:** Multiple identical blocks. Similar to encoder but with one key difference: masked attention (can't attend to future tokens). Allows autoregressive generation (one token at a time).

**Encoder-Decoder Attention:** Decoder attends to encoder outputs. Allows decoder to use input information. Seq2seq tasks (translation, summarization).

## 7.3 Encoder vs Decoder vs Encoder-Decoder

| Type                         | Attention                               | Causal Masking         | Use Cases                         | Examples            |
|------------------------------|-----------------------------------------|------------------------|-----------------------------------|---------------------|
| Encoder<br>(Bidirectional)   | Self-att (both directions)              | No                     | Understanding, classification     | BERT, RoBERTa       |
| Decoder<br>(Autoregressive)  | Masked self-att (left-to-right)         | Yes (can't see future) | Generation, next-token prediction | GPT-2, GPT-3, LLaMA |
| Encoder-Decoder<br>(Seq2seq) | Enc: bidirectional, Dec: autoregressive | Yes (encoder)          | Translation, summarization        | T5, mBART, BART     |

## 8. BERT & Encoder Models

### 8.1 What Is BERT?

Bidirectional Encoder Representations from Transformers (2018). A pre-trained transformer encoder. Trained on massive text corpus using self-supervised learning. Can see context from both directions (left and right). Use for classification, NER, Q&A;

### 8.2 Pre-training: Two Tasks

**Masked Language Modeling (MLM):** Randomly mask 15% of tokens, predict them from context. Forces the model to learn bidirectional context. "The [MASK] in the tree" → predict "cat".

**Next Sentence Prediction (NSP):** Given two sentences, predict if they are consecutive (classification task). Helps model understand sentence relationships. Used for pair tasks (Q&A, similarity).

### 8.3 Fine-Tuning BERT

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

# Load pre-trained BERT
model_name = 'bert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

# Fine-tune for sentiment classification
texts = ["I love this!", "This is terrible", ...]
labels = [1, 0, ...]

inputs = tokenizer(texts, padding=True, truncation=True, return_tensors='pt')
# inputs['input_ids'], inputs['attention_mask']

# Forward pass
outputs = model(**inputs, labels=torch.tensor(labels))
loss = outputs.loss

# Backward + optimize
loss.backward()
optimizer.step()

# Inference
with torch.no_grad():
    inputs_new = tokenizer("Great movie!", return_tensors='pt')
    outputs = model(**inputs_new)
    logits = outputs.logits
    prediction = torch.argmax(logits, dim=-1) # class 0 or 1
```

### 8.4 BERT Variants & Other Encoders

| Model | Key Difference | Use When |
|-------|----------------|----------|
|-------|----------------|----------|

|                   |                                                        |                                          |
|-------------------|--------------------------------------------------------|------------------------------------------|
| RoBERTa           | Better training (more data, longer training)           | Better long-range dependencies than BERT |
| ALBERT            | Lightweight (parameter sharing across layers)          | Resource-constrained (mobile, edge)      |
| DeBERTa           | Disentangled attention (separate position and content) | Needn't sacrifice performance            |
| ELECTRA           | Replaced token detection (GAN-like pre-training)       | Pushing, high performance                |
| multilingual-BERT | 101 languages                                          | Multilingual tasks                       |

## 9. GPT & Decoder Models (The LLM Era)

### 9.1 The GPT Family

Generative Pre-trained Transformer. Autoregressive decoder-only models. Trained to predict next token given previous tokens. This simple objective is incredibly powerful.

| Model   | Year | Parameters | Key Innovation                         | Access            |
|---------|------|------------|----------------------------------------|-------------------|
| GPT     | 2018 | 117M       | First large-scale generative model     | OpenAI paper      |
| GPT-2   | 2019 | 1.5B       | Showed zero-shot generalization        | Open source       |
| GPT-3   | 2020 | 175B       | Few-shot learning, in-context learning | API               |
| GPT-3.5 | 2022 | unknown    | Chat tuning, released as ChatGPT       | Free tier         |
| GPT-4   | 2023 | unknown    | Multimodal, better reasoning           | API, subscription |

### 9.2 Training: Next Token Prediction

```
# Example: "The cat sat on the mat"
# Token sequence: [the, cat, sat, on, the, mat]

# Training objectives:
# Input: [the, cat, sat, on, the] → Predict: mat
# Input: [the, cat, sat, on] → Predict: the
# Input: [the, cat, sat] → Predict: on
# ... (autoregressive: predict each token from previous ones)

# Loss = -log P(mat | the, cat, sat, on, the)
#       + -log P(the | the, cat, sat, on)
#       + ... (sum over all positions)

# This simple loss trains the model to learn language structure, semantics, facts, reasoning!
```

### 9.3 Inference: Generation

```
# Greedy decoding (pick highest probability token at each step)
prompt = "Once upon a time"
for i in range(100): # generate 100 tokens
    tokens = tokenizer(prompt)['input_ids']
    logits = model(tokens)[-1, :] # logits for last token
    next_token = argmax(logits)
    prompt += tokenizer.decode([next_token])

# Better: nucleus sampling (top-p sampling)
# Rather than argmax, sample from top-p tokens
# More diverse, less repetitive

# Or: temperature sampling
# softmax(logits / temperature)
# T=0.1: sharp distribution (greedy)
# T=1.0: standard softmax
# T=2.0: flat distribution (random)
```

## 9.4 Other Decoder Models

**LLaMA (Meta 2023):** Open-source LLM. Better efficiency than GPT-3. Sparked open-source LLM revolution.

Fine-tuned variants: Alpaca, Vicuna, Mistral

**Mistral (2023):** Smaller, faster, competitive performance. Better than LLaMA on many benchmarks.

**Gemini (Google 2023):** Multimodal (text, images, audio). Competitive with GPT-4. State-of-the-art on many benchmarks.

# 10. Hugging Face Ecosystem

## 10.1 What Is Hugging Face?

The go-to platform for NLP. Open-source transformers library (TensorFlow + PyTorch). Model Hub (millions of pre-trained models). Datasets library for download + preprocessing. Industry standard.

## 10.2 Core Components

```
# 1. Transformers Library (https://huggingface.co/transformers/)
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipeline

# 2. AutoModel (auto-detect architecture)
model_name = 'bert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)

# 3. Pipelines (high-level API)
classifier = pipeline('text-classification', model='bert-base-uncased')
result = classifier('I love transformers!')
# [{'label': 'POSITIVE', 'score': 0.999}]

# 4. Datasets Library
from datasets import load_dataset
dataset = load_dataset('glue', 'sst2')
# Automatic download, caching, formatting

# 5. Model Hub
# Browse at https://huggingface.co/models
# Filter by task, language, model type
# Download 1-liner: AutoModel.from_pretrained('model-name')
```

## 10.3 Tokenizers: BPE, WordPiece, SentencePiece

| Tokenizer                   | Method                                 | Vocabulary Size | Used In       | Pros/Cons                      |
|-----------------------------|----------------------------------------|-----------------|---------------|--------------------------------|
| WordPiece                   | Learned subword units via forward pass | 30k (BERT)      | BERT, RoBERTa | Balanced. Efficient.           |
| BPE<br>(Byte Pair Encoding) | Iterative merge of frequent byte pairs | 50k-100k (GPT)  | GPT-2, GPT-3  | Simple. Good compression.      |
| SentencePiece               | Language-agnostic, character-level     | 8k-20k (start)  | mBERT, T5     | Handles Unicode. Multilingual. |

## 11. Fine-Tuning & Practical Workflows

### 11.1 Full Fine-Tuning vs Parameter-Efficient Methods

| Approach                      | What's Updated                                 | Memory | Speed  | Performance    | When to Use                     |
|-------------------------------|------------------------------------------------|--------|--------|----------------|---------------------------------|
| Full Fine-tune                | All weights                                    | High   | Slow   | Best           | Large GPU, unlimited budget     |
| LoRA<br>(Low-Rank Adaptation) | Add small trainable matrices, freeze originals | Medium | Medium | 97-99% of full | Default choice now              |
| QLoRA                         | LoRA + 4-bit quantization                      | Low    | Fast   | 98%+ of full   | Limited GPU (e.g., 1x RTX 4090) |
| Adapters                      | Small modules between layers                   | Low    | Fast   | 95-98%         | Modular, multi-task             |

### 11.2 LoRA Example (Recommended)

```
from peft import get_peft_model, LoraConfig, TaskType
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

# Load model
model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-2-7b')
tokenizer = AutoTokenizer.from_pretrained('meta-llama/Llama-2-7b')

# Configure LoRA
lora_config = LoraConfig(
    r=8, # rank of LoRA matrices
    lora_alpha=16,
    target_modules=['q_proj', 'v_proj'], # which layers to add LoRA to
    lora_dropout=0.05,
    bias='none',
    task_type=TaskType.CAUSAL_LM
)

# Wrap model with LoRA
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 6,738,415,616 (0.06%)
# 99.94% of weights frozen!

# Train as usual (only LoRA matrices update)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
# ... training loop
```

### 11.3 Instruction Tuning

After pre-training, fine-tune on instruction-following examples: (instruction, input, output) triples. "Summarize this text: [text] → [summary]". This turns a language model into a helpful assistant. ChatGPT is GPT-3.5 + instruction tuning + RLHF.

### 11.4 RLHF (Reinforcement Learning from Human Feedback)

Train a reward model on human preferences (which outputs are better?). Then use RL to optimize model outputs for high reward. Makes models safer and more helpful. Complex but increasingly

common. DPO (Direct Preference Optimization) is simpler alternative.

## 12. Common Mistakes

**Not preprocessing/normalizing text:** Remove URLs, lowercase, handle special chars. Inconsistent input → poor results.

**Using wrong tokenizer:** Each model has its own tokenizer. Don't use BERT's tokenizer with GPT model. Mismatch causes poor performance.

**Truncating text naively:** If text exceeds `max_length`, don't just cut off. Use sliding window or summarize. Information loss.

**Not using attention masks:** If batch has variable lengths, pad and use `attention_mask`. Without it, model attends to padding tokens.

**Overfitting on small datasets:** Fine-tuning on 100 examples? You'll overfit. Use dropout, early stopping, aggressive regularization.

**Not using validation loss correctly:** Monitor validation loss, not training loss. Stop when val loss stops improving (early stopping).

**Using too high learning rate for fine-tuning:** Pre-trained weights already good. Start with `lr=1e-5` or `5e-5`, not `1e-3`.

**Ignoring class imbalance:** If dataset is 95% positive, 5% negative, model will just predict positive always. Use weighted loss or resampling.

**Not evaluating on test set:** Always hold out test set. Evaluate on it only once. Don't tune based on test results.

**Greedy decoding for generation:** Greedy (`argmax`) → repetitive, poor quality. Use nucleus sampling, beam search, or temperature for diversity.

## 13. Quick-Fire Q&A;

### [Easy] What's the difference between TF-IDF and word embeddings?

TF-IDF: sparse, frequency-based, no semantic meaning. Embeddings: dense, learned, capture similarity (cat  $\approx$  dog).

### [Easy] Why is BERT bidirectional but GPT is autoregressive?

BERT: understands context left+right (good for classification). GPT: generates left-to-right (can't peek at future). Different goals.

### [Medium] Explain attention in one sentence.

Model learns which input tokens are relevant for each output token, weights them, computes weighted sum.

### [Medium] What's the computational cost of transformers vs RNNs?

RNN:  $O(\text{seq\_len})$  sequential (can't parallelize). Transformer:  $O(\text{seq\_len}^2)$  for attention but parallelizable. For long sequences, Transformers may use sparse attention.

### [Medium] When do you use BERT vs GPT?

BERT: classification, NER, Q&A; (understanding). GPT: generation, completion, chat (production).

### [Hard] What's the difference between causal and bidirectional attention?

Causal: can't attend to future (masked, autoregressive). Bidirectional: can attend both ways (encoder, classification).

### [Medium] How do you fine-tune a pre-trained model?

Load pre-trained weights. Replace final layer for your task. Train with low learning rate ( $1e-5$ ). Use validation set for early stopping.

### [Medium] What is instruction tuning?

Fine-tune on (instruction, input, output) examples. Teaches model to follow instructions, not just predict next token.

### [Easy] Why use Hugging Face?

Pre-trained models, simple API, Model Hub, active community. Industry standard. Don't reinvent the wheel.

### [Medium] How do you handle long sequences?

Truncate intelligently or chunk. Use sparse attention (Longformer, BigBird). Fine-tune with longer max\_length if GPU allows.

## 14. One-Line Cheat Sheet

- Text preprocessing — Lowercase, remove URLs/punctuation, tokenize, remove stopwords, lemmatize.
- BoW — Represent text as word frequency vector. Simple, interpretable, no semantic meaning.
- TF-IDF — Weight words by importance. Common words (the) get low weight, rare words (specific) get high weight.
- Word2Vec — Learn dense embeddings where similar words are close. Two methods: CBOW (context→word) and Skip-gram (word→context).
- GloVe — Matrix factorization + word co-occurrence. Often better than Word2Vec. Free pre-trained vectors.
- FastText — Word2Vec with subword embeddings. Handles morphology and OOV words well.
- Attention — Weights relevance of input tokens for each output.  $\text{Query} \cdot \text{Key}^T$  gives attention scores.
- Self-attention — Token attends to all other tokens. Base of transformers. Multi-head: multiple perspectives simultaneously.
- Transformer — Attention + feedforward + layer norm + residuals. No RNNs. Parallelizable. Foundation of modern NLP.
- Positional encoding — Add position information to embeddings. Attention has no notion of position otherwise.
- BERT — Bidirectional encoder. Pre-trained via MLM + NSP. Fine-tune for classification, NER, Q&A.;
- GPT — Autoregressive decoder. Trained via next-token prediction. Fine-tune or prompt for generation.
- RoBERTa — Improved BERT. Better training, more data, longer training, better hyperparams.
- T5 — Encoder-decoder. Treat all NLP as text-to-text. Seq2seq: translation, summarization.
- Fine-tuning — Load pre-trained weights, replace final layer, train with low lr. Transfer learning is powerful.
- LoRA — Low-rank adaptation. Freeze base model, add small trainable matrices. 99.94% efficiency of full fine-tuning.
- Hugging Face — Industry standard. Transformers library, Model Hub, Datasets library.
- Tokenization — Convert text to integers. WordPiece (BERT), BPE (GPT), SentencePiece (T5, mBERT).
- Instruction tuning — Fine-tune on (instruction, input, output) examples. Teaches following instructions, not just predicting.
- Generation — Use beam search or nucleus sampling, not greedy. Better quality, more diverse.

## Final Note

NLP is in an exciting inflection point. Transformers have made the field incredible: ChatGPT, code generation, multimodal models. For placements in 2025-2026, you **MUST** understand transformers deeply. Know how BERT differs from GPT. Know why attention is powerful. Know how to fine-tune or prompt. Build a project: sentiment analysis with BERT, text generation with GPT, RAG chatbot with embeddings. Stay updated—the field moves incredibly fast. Read papers on arXiv. Follow industry leaders. You're living in the LLM era.